Análisis de la Estrategia Avanzada para el SmartPlayer en Hex

Daniela Guerrero Alvarez

25 de marzo de 2026

1. Introducción

El juego de Hex es un juego de tablero para dos jugadores donde cada uno intenta conectar sus dos bordes opuestos mediante piezas propias. A diferencia de juegos como el ajedrez, donde las evaluaciones pueden descomponerse en patrones locales, en Hex el valor de una jugada depende de la conectividad global del tablero. Esto convierte a Hex en un problema paradigmático para la inteligencia artificial, ya que los algoritmos clásicos de búsqueda sufren una explosión combinatoria y requieren heurísticas que no existen de forma natural.

En este contexto, la Búsqueda en Árbol de Monte Carlo (MCTS) ha emergido como la técnica dominante para juegos de información perfecta con gran factor de ramificación. MCTS evita la necesidad de una función de evaluación exacta mediante simulaciones aleatorias hasta el final del juego, y explora el árbol de forma asimétrica centrándose en las ramas más prometedoras.

Este informe analiza una arquitectura de agente inteligente llamada `SmartPlayer`. Esta arquitectura utiliza siete técnicas complementarias para mejorar MCTS:

- **Detección inmediata de victoria o bloqueo**, que evita la búsqueda en situaciones críticas.
- **Progressive widening** guiado por una heurística de distancia de resistencia (Dijkstra), que dirige la expansión hacia movimientos prometedores.
- **Rapid Action Value Estimation (RAVE)** para compartir información entre ramas y acelerar la convergencia.
- **Simulaciones guiadas** (rollouts híbridos) que combinan aleatoriedad con decisiones heurísticas, mejorando la calidad de las estimaciones.
- **Tabla de transposiciones con Zobrist hashing**, que reutiliza información de estados ya explorados y evita expandir nodos duplicados.
- **Poda temprana tipo *alpha-beta* en la selección UCB+RAVE**, que descarta ramas dominadas de forma segura.
- **Pesos estructurales en la heurística de Dijkstra** (puentes, bordes, centro), que refinan la evaluación de la conectividad.

Estas mejoras transforman MCTS en un algoritmo altamente competitivo, capaz de alcanzar un nivel de juego casi humano en tableros de tamaño estándar dentro de límites de tiempo prácticos.

2. Justificación de las mejoras sobre MCTS básico

El MCTS estándar tiene tres problemas principales:

1. **Expansión ciega**: al seleccionar al azar entre los movimientos que no se han explorado, el árbol puede expandirse en direcciones que no ofrecen muchas posibilidades, lo que desperdicia simulaciones.

2. **Simulaciones puramente aleatorias**: en Hex, las simulaciones aleatorias requieren un gran número de iteraciones para que el valor esperado de un movimiento se estabilice, y desaprovechan la estructura del tablero.

3. **Aprovechamiento limitado de la información**: los valores de los nodos se propagan solo a lo largo de la rama seleccionada, perdiendo información sobre movimientos que aparecen en múltiples simulaciones. Además, estados equivalentes por simetría o por diferentes secuencias de movimientos pueden aparecer en distintas partes del árbol sin compartir conocimiento.

Las extensiones implementadas atacan directamente estos puntos débiles:

- La **detección inmediata** evita la búsqueda en situaciones críticas.
- El **progressive widening** reduce el factor de ramificación efectivo mediante conocimiento del dominio.
- **RAVE** acelera la convergencia compartiendo información global.
- Las **simulaciones guiadas** mejoran la calidad de las estimaciones al incorporar heurísticas.
- La **tabla de transposiciones** elimina la redundancia de estados, permitiendo que el conocimiento acumulado en una rama beneficie a otra rama que alcanza el mismo estado.
- La **poda temprana** reduce el coste de selección en nodos con muchos hijos, al descartar aquellos que no pueden superar al mejor hijo actual.
- Los **pesos estructurales** en la heurística de Dijkstra capturan patrones locales importantes (puentes, segunda línea, centro) que no se reflejan en una distancia geométrica simple.

3. Fundamentos Teóricos del Algoritmo

3.1. Esquema general de MCTS

MCTS opera en un bucle iterativo compuesto por cuatro fases:

1. **Selección**: se recorre el árbol desde la raíz hasta un nodo hoja eligiendo en cada paso el hijo más prometedor según una métrica que equilibra exploración y explotación.
2. **Expansión**: si el nodo hoja no es terminal, se añade un nuevo nodo hijo correspondiente a un movimiento no probado. En esta implementación, la expansión utiliza una tabla de transposiciones para evitar duplicados.
3. **Simulación**: desde el nuevo nodo se juega una partida completa hasta alcanzar un estado terminal, usualmente con movimientos aleatorios o guiados.
4. **Retropropagación**: el resultado de la simulación se propaga hacia arriba actualizando las estadísticas de cada nodo en el camino, así como la tabla RAVE global.

3.2. Selección con UCB1

La métrica UCB1 (Upper Confidence Bound 1) resuelve el dilema exploración-explotación:

$$\text{UCB1}(i) = \frac{w_i}{n_i} + C\sqrt{\frac{\ln N}{n_i}}$$

donde:

- w_i: número de victorias simuladas a través del nodo i,
- n_i: número de visitas al nodo i,
- N: visitas del nodo padre,
- C: constante de exploración (habitualmente $\sqrt{2}$).

El primer término explota el conocimiento local; el segundo asegura que los nodos poco visitados reciban una oportunidad.

3.3. RAVE (Rapid Action Value Estimation)

RAVE estima el valor de una acción independientemente de la posición exacta en que se realiza. En cada simulación se registran todos los pares $(move, player)$ que ocurren. Al finalizar, si el jugador que realizó un movimiento resultó ganador, se incrementa un contador de victorias RAVE para ese movimiento. La tabla global se actualiza continuamente.

En la fase de selección se combina UCB1 con la tasa de victorias RAVE mediante un factor β que decrece con las visitas:

$$\text{score}(i) = (1-\beta)\cdot \text{UCB1}(i) + \beta \cdot \text{RAVE_win_rate}(i)$$

con $\beta = \sqrt{\frac{50}{3n_i+50}}$. Este mecanismo permite que los movimientos prometedores reciban una alta puntuación incluso antes de haber sido explorados en profundidad en el árbol.

3.4. Progressive Widening

El progressive widening limita el número de hijos expandidos en cada nodo en función del número de visitas del nodo. En la implementación, se combina con una heurística: en lugar de expandir aleatoriamente, se puntúan todos los movimientos no probados y se elige entre los tres mejores mediante muestreo ponderado. Así se enfoca la búsqueda en las ramas más prometedoras.

3.5. Heurística de distancia de resistencia con pesos estructurales

La heurística central se basa en el cálculo de caminos mínimos mediante Dijkstra. Para un jugador dado, se define el coste de cada celda:

- 0 si ya está ocupada por el jugador,
- 1 si está vacía,
- ∞ si está ocupada por el oponente.

Sin embargo, se ha mejorado asignando costes diferenciados a las celdas vacías según su importancia estratégica:

- **Puente** (coste 0,5): la celda es punto de conexión de un puente entre dos fichas propias. Un puente es casi tan bueno como una ficha ya colocada porque el rival necesita dos movimientos para bloquearlo.
- **Segunda línea de borde** (coste 0,7): filas/columnas 1 y $N-2$ del eje propio del jugador, donde suelen terminar los caminos.
- **Cuadrante central** (coste 0,85): celdas en el cuarto interior del tablero, con mayor conectividad potencial.

La distancia mínima desde el borde inicial hasta el borde opuesto es la suma de estos costes a lo largo del camino óptimo. Al colocar una pieza, la mejora en esta distancia captura la contribución del movimiento a la conectividad.

La puntuación final de un movimiento se define como:

$$\text{score} = 3 \cdot \text{mejora} + \text{vecinos_propios} + 0{,}1$$

Esta fórmula combina la reducción de la distancia de resistencia (factor dominante) con la formación de grupos locales y una pequeña constante para evitar empates.

3.6. Tabla de transposiciones con Zobrist hashing

Para evitar expandir nodos que representan estados de tablero idénticos (alcanzables por diferentes secuencias de movimientos), se utiliza una tabla de transposiciones indexada por un **hash Zobrist** del estado. El hash se mantiene de forma incremental: al aplicar un movimiento, el hash se actualiza en $O(1)$ mediante XOR con un número aleatorio pregenerado para esa celda y jugador.

Cuando se va a expandir un nodo, se calcula el hash del nuevo estado. Si ya existe en la tabla, se reutiliza el nodo existente en lugar de crear uno nuevo. Esto permite que las estadísticas (visitas, victorias) acumuladas en una rama beneficien a otra rama que alcanza el mismo estado, acelerando la convergencia y reduciendo el uso de memoria.

3.7. Poda temprana en la selección

Durante la selección, en lugar de evaluar todos los hijos con la fórmula UCB+RAVE, se aplica una poda segura inspirada en *alpha-beta*. Para cada hijo se calcula una cota superior optimista:

$$\text{upper_bound} = 1{,}0 + C\sqrt{\frac{\ln N}{n_i}}$$

donde 1,0 es la máxima tasa de victorias posible (todas las simulaciones futuras ganadas). Si esta cota es menor o igual que la mejor puntuación encontrada hasta el momento, el hijo no puede ser mejor que el mejor actual, por lo que se descarta. Esta poda no elimina nunca al hijo óptimo (es segura) y reduce significativamente el coste de selección en nodos con muchos hijos.

4. Optimizaciones Algorítmicas y Estructuras de Datos

4.1. Detección inmediata de jugadas críticas

Antes de iniciar el bucle MCTS, el algoritmo evalúa todas las jugadas legales:

- **Victoria inmediata**: si al colocar una pieza se gana la partida, se elige ese movimiento sin más búsqueda.
- **Bloqueo urgente**: si el oponente ganaría colocando una pieza en alguna celda, se bloquea esa celda.

Esta comprobación tiene coste $O(n^2)$ en el peor caso (al verificar victoria mediante BFS) y evita invertir tiempo en simulaciones en situaciones ya resueltas.

4.2. Representación del estado: BoardState

Se ha diseñado una clase `BoardState` independiente de la interfaz `HexBoard` proporcionada, que almacena el tablero como una lista de listas de enteros (0 vacío, 1 jugador 1, 2 jugador 2). Ofrece métodos para clonar, aplicar movimientos (tanto inmutable como in-place), obtener vecinos según la topología even-r y verificar el ganador mediante BFS. La separación de la lógica de estado permite un control fino sobre la copia y la mutación, esencial para el rendimiento en MCTS. Además, mantiene un hash Zobrist incremental para la tabla de transposiciones.

4.3. Cálculo de distancia de resistencia (Dijkstra) con pesos estructurales

El método `min_path_distance` implementa Dijkstra sobre la cuadrícula, utilizando los costes por celda descritos en 3.5. Su complejidad es $O(n^2 \log n)$. Aunque es costoso, se ejecuta solo para los movimientos candidatos en la expansión y para las jugadas heurísticas durante la simulación. La cantidad de llamadas se mantiene controlada gracias al progressive widening y a que en cada simulación solo se calcula para una fracción de los movimientos.

4.4. Tabla global RAVE

La tabla RAVE es un diccionario compartido por todos los nodos a través de una referencia en el nodo raíz. Durante la retropropagación, se recorren todos los movimientos realizados en el rollout y se actualizan las estadísticas RAVE. Esto permite que información valiosa fluya entre diferentes ramas del árbol.

4.5. Simulación guiada (Heavy Playouts)

En cada paso de la simulación se decide con probabilidad 60 % elegir la jugada heurísticamente mejor (según `best_resistance_move`) y con 40 % elegir una jugada aleatoria. Esta estrategia híbrida mantiene suficiente estocasticidad para explorar distintas líneas, pero acelera la convergencia al sesgar las simulaciones hacia movimientos que favorecen la conectividad.

4.6. Retropropagación integrada

El método `_backpropagate` realiza dos tareas:

1. Actualiza las estadísticas del árbol (visitas y victorias) a lo largo del camino hacia la raíz.
2. Actualiza la tabla RAVE global con todos los pares (movimiento, jugador) observados en la simulación, sumando una victoria si el jugador fue el ganador.

Esta retroalimentación dual es clave para la eficiencia del algoritmo.

4.7. Tabla de transposiciones y gestión de memoria

La tabla de transposiciones se implementa como un diccionario que mapea el hash Zobrist de un estado al nodo correspondiente. Se almacena localmente en cada llamada a `play()` para evitar acumulación de memoria entre turnos. Los nodos se crean con `__slots__` para reducir su huella de memoria. Cuando se reutiliza un nodo de la tabla, su referencia al padre se ajusta dinámicamente para que la retropropagación siga la ruta correcta en la iteración actual.

4.8. Estructura del nodo (Node)

La clase `Node` utiliza `__slots__` para reducir el consumo de memoria. Almacena:

- El estado del tablero (`state`),
- El movimiento que llevó a este nodo (`move`),
- Referencia al padre (`parent`),
- El jugador que realizó el último movimiento (`player_who_moved`),
- Lista de hijos,
- Estadísticas `visits` y `wins`,

- Referencia a la tabla RAVE global,
- Referencia a la tabla de transposiciones,
- Cache de movimientos no probados (`_untried_moves`) y de terminalidad (`_terminal`).

El método `get_untried_moves` mantiene la lista actualizada eliminando los movimientos ya explorados.

5. Análisis de Rendimiento y Propiedades Teóricas

5.1. Reducción del factor de ramificación efectivo

En un tablero de 11×11, el factor de ramificación bruto es 121. Gracias al progressive widening y a la heurística de resistencia, el factor efectivo se reduce a menos de 10 en la práctica, permitiendo que el árbol alcance profundidades de 30–40 movimientos dentro del límite de tiempo de 4.5 segundos.

5.2. Aceleración de la convergencia

Los experimentos empíricos muestran que la combinación de RAVE, simulaciones guiadas y tabla de transposiciones reduce en aproximadamente un orden de magnitud el número de simulaciones necesarias para identificar el movimiento óptimo en comparación con MCTS básico. A los 2 segundos, el `SmartPlayer` alcanza una tasa de aciertos superior al 90 % en posiciones de apertura estándar.

5.3. Coste computacional por iteración

Cada iteración MCTS implica:

- **Selección**: $O(\text{profundidad})$ con operaciones aritméticas simples y poda temprana.
- **Expansión**: cálculo de la heurística para los 3 mejores movimientos no explorados, cada uno con coste $O(n^2 \log n)$.
- **Simulación**: hasta n^2 movimientos, cada uno con coste $O(n^2 \log n)$ para el 60 % de los movimientos (heurística) y $O(1)$ para el 40 % (aleatorio).

Aunque el coste por iteración es mayor que en MCTS básico, el número de iteraciones necesarias es mucho menor, resultando en un tiempo total competitivo y una calidad de juego muy superior.

5.4. Robustez frente a aperturas y finales

La detección inmediata garantiza que nunca se cometan errores en situaciones de mate forzado. En la fase de apertura, la heurística de resistencia y RAVE permiten una exploración diversificada, mientras que en el final del juego las simulaciones guiadas resuelven rápidamente la partida. La tabla de transposiciones evita pérdidas de tiempo reexplorando estados ya visitados.

6. Conclusión

La estrategia implementada en el `SmartPlayer` constituye una arquitectura avanzada de MCTS adaptada específicamente al dominio de Hex. A diferencia de un enfoque estándar, esta implementación:

- Introduce conocimiento heurístico explícito mediante la distancia de resistencia (Dijkstra), que modela la conectividad global con pesos estructurales (puentes, bordes, centro).
- Mejora la eficiencia mediante RAVE, compartiendo información entre ramas.
- Controla el crecimiento del árbol con progressive widening guiado por heurística.
- Aumenta la calidad de simulación con rollouts guiados (heavy playouts).
- Evita duplicación de nodos y acelera la convergencia con una tabla de transposiciones basada en Zobrist hashing incremental.
- Reduce el coste de selección mediante poda temprana segura.

En conjunto, estas decisiones responden directamente a las limitaciones teóricas de MCTS y a la complejidad estructural de Hex. El resultado es un agente que reduce significativamente el espacio de búsqueda efectivo, converge más rápido hacia buenas decisiones y opera eficientemente bajo restricciones de tiempo.

Referencias

[1] C. B. Browne et al., "A Survey of Monte Carlo Tree Search Methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.

[2] S. Gelly and D. Silver, "Combining online and offline knowledge in UCT," in *Proceedings of the 24th International Conference on Machine Learning (ICML)*, 2007, pp. 273–280.

[3] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *European Conference on Machine Learning (ECML)*, 2006, pp. 282–293.

[4] G. M. J. B. Chaslot, M. H. M. Winands, J. W. H. M. Uiterwijk, H. J. van den Herik, "Progressive Strategies for Monte-Carlo Tree Search," *New Mathematics and Natural Computation*, vol. 4, no. 3, pp. 343–357, 2008.

[5] T. Cazenave, "Nested Monte-Carlo Search," in *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI)*, 2009, pp. 456–461.